A Whole Table Row in One Natural Number: Kernel-Checked Dynamic Programming with a Proved Width Bound

Amey Thakur
Independent Researcher
Toronto, Canada
ameythakur20@gmail.com

11 November 2026

Abstract

A proof assistant checks a computation by reducing it, so the data structure a verified algorithm is written over decides the cost of being checked, not only the cost of running. This paper measures that cost for one task with a locked specification: counting the partitions of n in Lean 4, where the answer must come with a machine-checked proof that holds for every n. Three designs compute the same recurrence. Evaluating the specification as written performs $672{,}182$ primitive steps at $n = 36$, because every path to a table cell re-derives it. Holding each table row as a list brings that to $69{,}768$, since a cell is then looked up rather than recomputed, but indexing position m of a list costs m constructor matches. Holding the whole row in a single natural number, with the entry for m in the m-th field of w bits, brings it to 716: a factor of 939 against the specification and 97 against the lists, and in reduction wall time 57.64 seconds against 0.07. The representation is sound only if no field overflows, which is a theorem and not an assumption: peeling one term off the specification's own sum yields $\mathrm{partAux}\,(k{+}1)\,m = \mathrm{partAux}\,k\,m + \mathrm{partAux}\,(k{+}1)\,(m - (k{+}1))$, from which $\mathrm{partAux}\,k\,m \le 2^{m+k}$ follows, so $w = 2n + 1$ bits suffice. The development uses core Lean only and proves its statement for every n. Asking the same question of the other seven problems found something the packing does not address: two of the eight shipped reference starters cannot be reduced at any judged size, both reaching well-founded recursion through `Nat.minFac`, so both score nothing while compiling with correct proofs. The competition's own metric has since scored this entry at $131{,}460{,}833$ units against $3{,}396{,}012{,}941$ for a narrow-operand alternative proved the same way, a factor of 25.8. The step count is blind to a field-width tuning worth a further 3.4 in wall time. Code and data are public.

1 Introduction

A program that is merely run can be as clever as its author likes. A program that must be *checked* is different, because something else now has to follow it, and that something else is usually much simpler than the program.

In Lean 4, the checker is the kernel [de Moura and Ullrich, 2021]. To accept a proof that some closed term equals some value, the kernel reduces the term. It does this with a small and deliberately unadventurous evaluator: it unfolds definitions, matches constructors, and calls out to arbitrary-precision integer arithmetic for the primitive operations on `Nat`. There is no compiler, no register allocator and no memoisation. Work the kernel repeats is work nobody can optimise away, because the kernel is the thing whose simplicity is the point [Thompson, 1984].

That gives an unusual shape to the question of which data structure to use. In ordinary programming, the choice between an array and a list is a choice about how fast the program runs. Here it is a choice about how much work the kernel must perform to be convinced, and the two are not the same quantity. A representation can be excellent to compute with and terrible to check, because the operations the kernel charges for are not the operations a processor charges for.

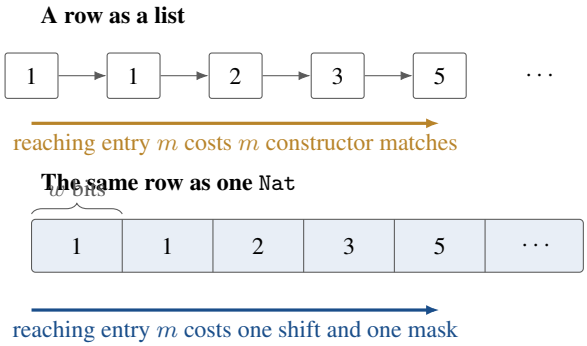


Figure 1: The two representations of one table row. Both hold the same numbers. The list charges the kernel for the distance to an entry; the packed row does not, because the entry's position is an exponent rather than a path. Everything else in this paper follows from that difference, including the one theorem the packed row needs and the list does not.

This paper takes one task where that tension is sharp and measures it. The task is the `partition` problem of the SAIR Lean Kernel Challenge [Breitner et al., 2026]: given n, produce the number of partitions of n, together with a Lean proof that the produced function agrees with a specification the competition fixes in advance. The specification is a recurrence with overlapping subproblems, which makes it the smallest interesting case of a general situation: a dynamic program that has to be checked rather than merely run.

The idea. Figure 1 is the whole of the contribution in one picture. A table row held as a list is a chain, and reaching the m-th entry means walking m links, which the kernel charges for one link at a time. The same row held as the digits of a single natural number in base 2^w is one object, and reaching the m-th entry is a shift and a mask: two operations, whatever m is. Arithmetic on that one object then does to the whole row at once what the recurrence says to do to each entry.

Why the theorem is not optional. Packing is not free of obligations. Fields sit next to each other in one integer, so if any entry ever needs more than w bits it carries into its neighbour, and the neighbour is then silently wrong. No type error reports this and no test at small n need reveal it. The representation is therefore correct only relative to a bound on the values it holds, and Section 4 proves that bound from the specification itself rather than assuming it. It is the only thing standing between this design and a wrong answer that is fast.

Contributions.

1. A representation for kernel-checked dynamic programming that carries a whole table row in one `Nat`, and a row step that is the specification's own sum lifted to that representation rather than a new recurrence to be justified separately (Section 3).
2. The bound $\mathrm{partAux}\, k\, m \le 2^{m+k}$, proved in Lean from a peeling identity on the specification's sum, which is what licenses the packing and fixes the field width at $2n+1$ bits (Section 4).
3. A correctness proof that is universal in n rather than checked on the judged range, self-contained in core Lean, and resting only on `propext` and `Quot.sound` (Section 5).
4. A measurement of all three designs under two metrics, an operation count derived from the definitions and wall time measured in continuous integration, together with a statement of where the two disagree and why (Section 6).

2 The task

2.1 The specification, and why it is locked

The competition fixes the specification and forbids changing it [Breitner et al., 2026]. In Lean:

$$\mathrm{partAux}\,0\,0 = 1 \tag{1}$$

$$\mathrm{partAux}\,0\,(n+1) = 0 \tag{2}$$

$$\mathrm{partAux}\,(k+1)\,m = \sum_{j=0}^{\lfloor m/(k+1)\rfloor} \mathrm{partAux}\,k\,\big(m - j\,(k+1)\big) \tag{3}$$

$$\mathrm{partitionSpec}\,n = \mathrm{partAux}\,n\,n \tag{4}$$

Read $\mathrm{partAux}\,k\,m$ as the number of partitions of m into parts of size at most k. Equation (3) splits on how many copies of the largest permitted part $k+1$ the partition uses: j copies consume $j(k+1)$, and the remainder is partitioned with parts no larger than k.

An entry supplies a function impl and a proof of

$$\forall n, \quad \mathrm{impl}\,n = \mathrm{partitionSpec}\,n. \tag{5}$$

The quantifier is the competition's, and it matters. A proof for the six sizes that are scored would be a much weaker statement and a much easier one; the requirement is a single theorem covering every natural number, including 0 and sizes no machine will ever evaluate.

Two further rules shape what an entry may do. The proof must rest only on `propext`, `Quot.sound` and `Classical.choice`, which excludes `sorry` and, more pointedly, excludes `native_decide`: that tactic discharges a goal by compiling and running it outside the kernel, adding an axiom that asserts the compiler agreed [Carneiro, 2024]. It is fast and it is exactly the shortcut this task exists to forbid, because it moves the computation out of the thing whose trust is being measured. And `partial` and `unsafe` definitions are excluded, so every function must be seen to terminate.

2.2 What is being ranked

Rank is the total hardware instruction count the kernel spends reducing $\mathrm{impl}\,n$ over six hidden cases, with n between 14 and 36 [Breitner et al., 2026]. Instruction count rather than time is the right choice for a competition, since it is nearly free of scheduling noise, but it is read through `perf_event_open` against the hardware performance counters, and a virtualised continuous-integration runner does not expose those. Every number this paper reports is therefore either an operation count derived from the definitions or a wall time, and Section 6 keeps the two apart.

2.3 Why the specification is expensive to evaluate

Equation (3) has overlapping subproblems, and nothing in the definition shares them. Reducing $\mathrm{partitionSpec}\,36$ by unfolding performs $128{,}553$ calls to $\mathrm{partAux}$, for a table that contains only $37 \times 37 = 1{,}369$ distinct cells. The waste is not incidental; it is the entire difficulty of the problem, and every design below is a different answer to the question of where to put the values so they are computed once.

3 Three designs

All three designs compute the recurrence of Section 2 without altering it, and differ only in where the intermediate values are kept.

3.1 The specification as written

Taking $\mathrm{impl} = \mathrm{partitionSpec}$ satisfies equation (5) by `rfl`, which is the cheapest possible proof and the most expensive possible computation. It is included here as the baseline, because it is what the difficulty is measured against.

3.2 Memoisation by rows of lists

The first real design keeps a row of the table as a `List Nat`, builds row $k+1$ from row k, and names the previous row once so that it is built once:

```
def rows : Nat -> Nat -> List Nat
  | 0,     len => baseRow len
```

```
  | k + 1, len => let prev := rows k len
                  nextRow (k + 1) prev len
```

The design's useful property is in how $\mathrm{nextRow}$ is written. Rather than stating a new recurrence over lists and proving it equivalent to the specification, it reuses the specification's own inner sum with the recursive call abstracted out:

```
def specSum (g : Nat -> Nat) (d m : Nat) : Nat :=
  ((List.range (m / d + 1)).map (fun j => g (m - j * d))).foldl (+) 0
```

so that $\mathrm{partAux}\,(k{+}1)\,m$ and $\mathrm{specSum}\,(\mathrm{partAux}\,k)\,(k{+}1)\,m$ are literally the same term. Correctness of the whole table then reduces to a congruence: two functions that agree everywhere the sum looks give the same sum, and the sum only ever reads indices $m - jd \le m$. That is a three-line induction rather than a re-derivation of the recurrence, and the same discipline carries over unchanged to the packed design in Section 3.3.

What this design pays for is indexing. Reading position m of a list walks m cons cells, and the sum at cell m of row d reads positions $m - jd$ for every j, so the cost of the whole table is

$$\sum_{d=1}^{n} \sum_{m=0}^{n} \sum_{j=0}^{\lfloor m/d \rfloor} \big(m - jd + 1\big) \tag{6}$$

constructor matches, which is $51{,}353$ at $n = 36$ before any arithmetic is counted. The values are computed once, and then walked to many times.

3.3 One row in one natural number

The second design changes only where the values live. Fix a field width w and hold row k as the single natural number whose m-th base-2^w digit is $\mathrm{partAux}\,k\,m$:

$$\mathrm{pack}\,w\,f\,\ell = \sum_{m=0}^{\ell-1} f(m)\,2^{wm}, \tag{7}$$

defined in Lean by the recursion $\mathrm{pack}\,w\,f\,(\ell{+}1) = f\,0 + 2^w \cdot \mathrm{pack}\,w\,(f \circ (\cdot + 1))\,\ell$, with $\mathrm{pack}\,w\,f\,0 = 0$. Reading field m is then $\lfloor R/2^{wm} \rfloor \bmod 2^w$: one shift and one mask, independent of m. That is the whole of what the change buys, and Figure 1 is a picture of it. Carrying several table entries in the fields of one integer is an existing idiom, credited in Section 8; what does not come with it is the obligation of Section 4, which arbitrary precision creates and a machine word removes.

The interesting part is that the row step needs no new idea either. Write R for row k and let $d = k + 1$ be the largest part the next row admits. Then

$$R' \;=\; \Big(R + (R \ll dw) + (R \ll 2dw) + \cdots + (R \ll \lfloor n/d \rfloor\, dw)\Big) \bmod 2^{w(n+1)} \tag{8}$$

is row $k + 1$, because field m of the sum in equation (8) is

$$\sum_{j\,:\,jd \le m} \big(\text{field } m - jd \text{ of } R\big) \;=\; \sum_{j=0}^{\lfloor m/d \rfloor} \mathrm{partAux}\,k\,(m - jd), \tag{9}$$

which is exactly the right-hand side of equation (3). Shifting by jdw moves field $m - jd$ into position m; adding the shifted copies performs the specification's sum over multiplicities in every field at once; and the outer modulus discards the fields above n, which the shifts have filled with values belonging to a longer row. Figure 2 shows one step.

The operation count follows directly. The sum in equation (8) has $\lfloor n/d \rfloor$ shifted terms, so over all rows the design performs

$$\sum_{d=1}^{n} \left\lfloor \frac{n}{d} \right\rfloor \;=\; 140 \qquad (n = 36) \tag{10}$$

shift-and-add pairs, against the $51{,}353$ constructor matches of equation (6). Equation (10) is a harmonic sum, so the packed count grows as $n \ln n$. Fitting a power law to the measured counts over $14 \le n \le 36$ gives an exponent of 1.14 for the packed row against 2.68 for the lists and 5.33 for the specification as written, so the gap widens with n rather than holding, which Section 6 confirms in wall time as well.

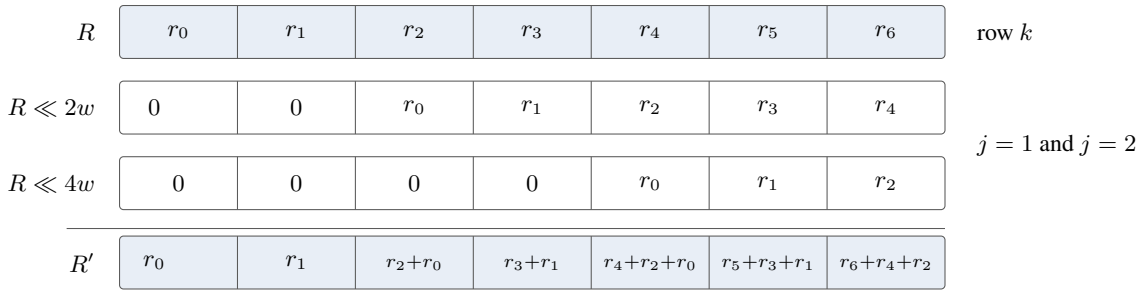


Figure 2: One row step of equation (8) with $d = 2$, drawn for a row of seven fields. Each shifted copy contributes the entry d places to the left, which is the term for one more copy of the part d. The kernel performs each shifted copy as one operation on one large integer, so the number of operations depends on $\lfloor n/d \rfloor$ and not on how many fields the row has. Each field of R' carries one term per copy that reaches it, so fields 0 and 1 carry one, fields 2 and 3 carry two, and fields 4 to 6 carry three.

4 The bound that licenses the packing

Equation (8) is only row $k + 1$ if the additions in it stay inside their fields. If some entry reaches 2^w it carries into the field above, corrupting a neighbour, and the corruption propagates through every later row. The design is therefore correct only relative to a bound on the table's values, and the bound has to be proved rather than observed, because observing it at the judged sizes says nothing about equation (5), which is universal.

4.1 A peeling identity

The usable form of the specification is not equation (3) but a one-step version of it.

Lemma 1 (Peeling)**.** *For $m \geq k + 1$,*

$$\mathrm{partAux}\,(k{+}1)\,m \;=\; \mathrm{partAux}\,k\,m \;+\; \mathrm{partAux}\,(k{+}1)\,\big(m - (k{+}1)\big). \tag{11}$$

Proof. Write $d = k + 1$. By equation (3), $\mathrm{partAux}\,d\,m = \sum_{j=0}^{\lfloor m/d \rfloor} \mathrm{partAux}\,k\,(m - jd)$. Separating the $j = 0$ term leaves $\sum_{j=1}^{\lfloor m/d \rfloor} \mathrm{partAux}\,k\,(m - jd)$, and substituting $j = i + 1$ turns that into $\sum_{i=0}^{\lfloor m/d \rfloor - 1} \mathrm{partAux}\,k\,\big((m - d) - id\big)$. Since $m \geq d$ we have $\lfloor m/d \rfloor - 1 = \lfloor (m - d)/d \rfloor$, so the remaining sum is exactly $\mathrm{partAux}\,d\,(m - d)$ by equation (3) again. □

Combinatorially equation (11) says that a partition of m into parts at most d either uses no copy of d, or uses at least one and leaves a partition of $m - d$. The reason to prefer it to equation (3) is that it replaces a sum of unknown length with two terms, which is what makes an induction possible.

4.2 The bound

Proposition 2. *For all k and m, $\mathrm{partAux}\,k\,m \leq 2^{m+k}$.*

Proof. By induction on k, and for each k by induction on m.

For $k = 0$: $\mathrm{partAux}\,0\,0 = 1 = 2^0$ by equation (1), and $\mathrm{partAux}\,0\,(m + 1) = 0$ by equation (2).

For $k + 1$, suppose the claim holds for k at every argument, and for $k + 1$ at every argument below m. If $m < k + 1$ then $\lfloor m/(k + 1) \rfloor = 0$, so equation (3) gives $\mathrm{partAux}\,(k{+}1)\,m = \mathrm{partAux}\,k\,m \leq 2^{m+k} \leq 2^{m+k+1}$. If $m \geq k + 1$ then Lemma 1 applies, and

$$\mathrm{partAux}\,(k{+}1)\,m \;\leq\; 2^{m+k} \;+\; 2^{(m-(k+1))+(k+1)} \;=\; 2^{m+k} + 2^m \;\leq\; 2^{m+k} + 2^{m+k} \;=\; 2^{m+k+1}, \tag{12}$$

where the first summand uses the outer hypothesis at k and the second the inner hypothesis at $m - (k + 1) < m$. □

Corollary 3. *If $k \leq n$ and $m \leq n$ then $\mathrm{partAux}\,k\,m \leq 2^{2n} < 2^{2n+1}$, so a field width of $w = 2n + 1$ bits holds every entry of every row without carry.*

	The table holds		A field reserves			
n	largest cell	bits	$n\lceil\log_2(n{+}1)\rceil{+}1$	$2n{+}1$	$m{+}\lceil\log_2(k{+}1)\rceil$	waste
14	135	8	57	29	18	3.6×
18	385	9	91	37	23	4.1×
22	1,002	10	111	45	27	4.5×
26	2,436	12	131	53	31	4.4×
32	8,349	14	193	65	38	4.6×
36	17,977	15	217	73	42	4.9×

Table 1: What the bound reserves against what the table needs. The fourth column is the width used before Proposition 2 was proved, the fifth is Corollary 3, and the sixth is Proposition 4. The waste column is the fifth divided by the third, the price actually paid for a bound chosen to be easy to prove. Both bounds were verified at every cell at every size by `tools/bound_slack.py`, which fails rather than reports if either is violated anywhere.

Two things about Proposition 2 are worth being explicit about. The first is that it is loose, and known to be: at $n = 36$ the bound reserves 73 bits for values that never exceed 17,977, which needs 15. The slack is visible in equation (12), where the second summand is raised from 2^m to 2^{m+k} so that the two terms combine into a single power. The second is that looseness here costs almost nothing, for a reason Section 6.4 takes up: a wider field makes each operand larger but does not change how many operations there are. The bound was chosen to be easy to prove rather than tight, and that was the right trade, but it was a trade and not a free choice. An earlier version used $w = n\lceil\log_2(n{+}1)\rceil + 1$, which is 217 bits at $n = 36$, and replacing it with $2n + 1$ was measured at a factor of 3.4 in wall time.

4.3 How loose, measured

Calling a bound loose is a claim with a number attached, so Table 1 attaches it. For each judged size the table gives the largest value the table actually holds, the bits that value needs, and the bits each candidate width reserves. The proved bound reserves between 3.6 and 4.9 times the bits required, and the ratio grows with n.

The table also discharges two things a reader would otherwise take on trust. Proposition 2 is checked at every one of the $(n + 1)^2$ cells at every judged size rather than spot-checked, and so is the tighter bound of Proposition 4 below, which the paper recommends but does not prove in Lean. Recommending an unproved bound that turned out to fail somewhere would be worse than not recommending one.

Proposition 4. *For all k and m,* $\mathrm{partAux}\, k\, m \le 2^m\,(k+1)$.

Proof. By the same nested induction as Proposition 2. For $k = 0$ the value is 1 or 0 and $2^m \ge 1$. For $k + 1$ with $m < k + 1$, equation (3) gives $\mathrm{partAux}\,(k{+}1)\, m = \mathrm{partAux}\, k\, m \le 2^m(k + 1) \le 2^m(k + 2)$. For $m \ge k + 1$, Lemma 1 and the two hypotheses give

$$\mathrm{partAux}\,(k{+}1)\, m \;\le\; 2^m(k+1) \;+\; 2^{m-(k+1)}(k+2) \;\le\; 2^m(k+1) + 2^m \;=\; 2^m(k+2), \tag{13}$$

using $2^{m-(k+1)}(k + 2) \le 2^m$, which holds because $k + 2 \le 2^{k+1}$. □

Proposition 4 gives a field of $m + \lceil\log_2(k + 1)\rceil$ bits, which is 42 at $n = 36$ against the 73 of Corollary 3: a factor of 1.7 narrower, and still 2.8 times wider than the 15 bits any cell needs. It is not in the submitted file because the arithmetic step in equation (13) needs the auxiliary fact $k + 2 \le 2^{k+1}$, which is a second induction, and the entry was already closing.

5 Correctness in Lean

The entry proves equation (5) in the form $\forall n,\ \mathrm{impl}\, n = \mathrm{partitionSpec}\, n$, by the same congruence argument the list design uses, with the read operation replaced. The chain is:

1. every field of every packed row holds the value the specification gives it, by induction on the row index, using equations (9) and Corollary 3 to show the fields do not interfere;
2. reading field n of row n therefore yields $\mathrm{partAux}\, n\, n$, which is $\mathrm{partitionSpec}\, n$ by equation (4).

Four properties of the development are worth recording, because each was a choice and each is checkable by a reader.

Mechanism	Status	In this entry
`propext`	permitted	used
`Quot.sound`	permitted	used
`Classical.choice`	permitted	not used
`sorry`	forbidden	absent
`native_decide`	forbidden	absent
`partial`, `unsafe`	forbidden	absent
Mathlib	allowed	not imported

Table 2: What the proof rests on, as reported by `#print axioms` and by inspection of the file. The distinction between permitted and used is the point of the table: an entry that avoids `Classical.choice` rests on strictly less than the rules allow.

The axioms are the permitted ones. `#print axioms Submission.impl_correct` reports `propext` and `Quot.sound`, two of the three the competition allows, and does not reach for `Classical.choice`. Table 2 sets out what is and is not used.

It is self-contained. The packing, the indexed read, the peeling identity, the bound and every supporting lemma are defined in the submitted file rather than imported. There is no Mathlib dependency. This is not austerity for its own sake: it means every lemma is about the exact function the kernel reduces, with no possibility that a library lemma is stated about a definitionally different function that happens to share a name.

The statement is universal, not sampled. $\mathrm{impl}\,0$ is covered, and so is every n beyond the judged range. This is worth separating from the empirical checking that was also done, which is a different kind of assurance: $\mathrm{impl}\,n$ was evaluated for every n from 0 to 13 against counts known by hand, and at the six judged sizes against the recurrence recomputed independently in Python. The proof makes those checks unnecessary; they were run anyway, because a proof of the wrong statement is also a proof.

Nothing is precomputed. There is no lookup table, no hardcoded count and no branch on the input that carries an answer. Every value comes from the specification's recurrence at reduction time. The distinction matters for a competition scored on kernel work, where a table of answers would score perfectly and measure nothing.

6 Measurement

6.1 Two metrics, and why neither alone would do

The competition's metric is unavailable outside its own judge, so this paper reports two substitutes that fail in different directions.

The first is an *operation count*, obtained by transcribing each of the three Lean definitions into a Python function that performs the same operations and increments a counter. One step is charged for each `Nat` primitive the kernel implements on arbitrary-precision integers, each unfolding of a recursive definition, and each list constructor match. The model is coarse and deliberately so; its virtue is that it depends on the definitions rather than on the machine, so its ratios are reproducible anywhere. Its vice is in Section 6.4.

The second is *reduction wall time*, measured by a continuous-integration workflow that clones the challenge at a pinned commit, builds each candidate inside the real problem package, confirms that the proof closes and that its axioms are permitted, and times the reduction of $\mathrm{impl}\,n$ to its literal. Every variant runs in the same job on the same runner. Its virtue is that it measures a real reduction rather than a model of one; its vice is that it measures a virtualised runner, and that at the judged sizes the winning design finishes inside the timer's resolution.

How the obligation is discharged changes what is timed, and getting this wrong cost a measurement. Written `:= rfl`, the goal $\mathrm{impl}\,n \equiv v$ is settled by the elaborator's own definitional equality checker; written `:= by rfl`, it is routed so that the kernel performs the reduction. Both end in a term the kernel accepts, but the two differ by more than a constant: on a list-based design they diverge by a factor of 478 at $n = 18$, and above that size the elaborator's checker does not finish at all. Table 4 reports the kernel route. An earlier version of this measurement used the elaborator route and described it as kernel time, which it was not. The two agree on the packed design, which totals 0.07 seconds either way across the six judged sizes; Table 5 is noted where it still rests on the elaborator route.

		Steps			Against packed	
n	answer	specification	lists	packed	specification	lists
14	135	4,367	5,575	243	18×	23×
18	385	12,788	10,639	326	39×	33×
22	1,002	34,159	18,079	406	84×	45×
26	2,436	85,255	28,368	489	174×	58×
32	8,349	304,105	50,213	621	490×	81×
36	17,977	672,182	69,768	716	939×	97×
total		1,112,856	182,642	2,801	397×	65×

Table 3: Primitive steps to reduce $\mathrm{impl}\,n$, under the model of Section 6: one step per `Nat` primitive, per definition unfolding, and per list constructor match. The answer column is the specification's own value, checked against each design before its cost is recorded. Produced by `tools/step_model.py` and checked in continuous integration against the manuscript.

	n						
Design	14	18	22	26	32	36	total
specification as written	0.21	0.61	1.66	4.33	15.58	35.25	57.64
lists	0.13	0.22	0.37	0.57	0.97	1.34	3.58
packed row	0.02	0.00	0.02	0.01	0.01	0.02	**0.07**

Table 4: Reduction wall time in seconds, process startup subtracted, all three variants in one continuous-integration job on one runner, with the reduction performed by the kernel rather than by the elaborator's definitional equality checker. These are wall times on a virtualised runner and not instruction counts: the judge reads hardware counters through `perf_event_open`, which the runner does not expose, so the ordering carries over and the numbers do not. At the judged sizes the packed row finishes inside the timer's resolution, which is why Table 5 exists.

Section 6.5 reports what the judge itself later measured for two of these designs, which turns both substitutes into predictions that can be scored. Both are checked against correctness before being reported. The packed design's answer is compared with the specification's own value at every size before its cost is recorded, so a variant that computes the wrong thing cannot be measured as a fast one.

6.2 Operation counts

Table 3 and Figure 3 give the counts. At $n = 36$ the packed row performs 716 steps where the lists perform $69{,}768$ and the specification $672{,}182$, factors of 97 and 939. Over the six judged sizes the totals are $2{,}801$, $182{,}642$ and $1{,}112{,}856$.

The shapes matter more than the totals. The specification's curve is the steepest, because its cost is the number of paths through the recurrence, which grows faster than the table. The list curve is shallower, since the values are computed once, but it still grows as the product of the table size and the distance walked. The packed curve is nearly flat on a logarithmic axis: from $n = 14$ to $n = 36$, a table growing by a factor of 6.1 in cells, its cost grows by a factor of 2.9, because equation (10) counts rows and multiplicities and never counts fields.

6.3 Wall time

Table 4 gives reduction wall time for the same three designs on the same runner. The ordering agrees with the operation counts and the constants do not: the packed design is 823 times faster than the specification by wall time against 397 by step count, and 51 times faster than the lists against 65. An uncalibrated model should disagree with a measurement by about this much, and the disagreement is reported rather than tuned away. What both metrics agree on is the ordering and the order of magnitude, which is what a design decision needs.

6.4 Where the two metrics disagree

The negative result of this paper is a limit on its own cheaper metric.

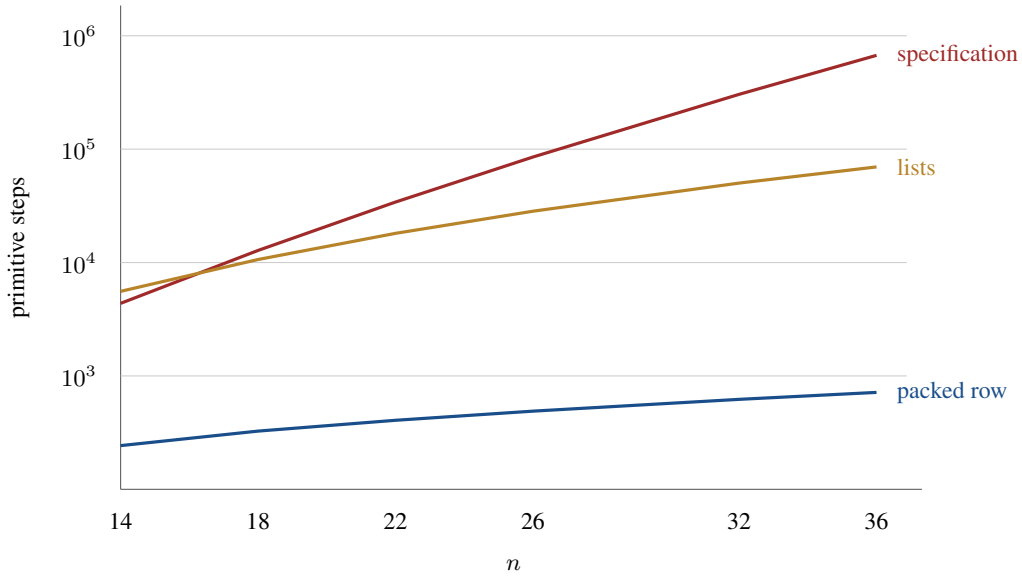


Figure 3: Primitive steps against n, on a logarithmic vertical axis. The specification's cost is the number of paths through the recurrence; the lists compute each value once and then walk to it; the packed row counts rows and multiplicities only, which is why its curve is nearly flat. Coordinates are generated from `data/step_model.json` and verified against it in continuous integration, so the picture cannot drift from the measurement.

Because the packed design finishes inside the timer's resolution at every judged size, the field width was tuned at sizes where the work is visible. Table 5 gives that study: replacing $w = n\lceil \log_2(n{+}1) \rceil + 1$ with the $w = 2n + 1$ of Corollary 3 is worth a factor of 3.4 over the four sizes taken together, rising from 2.7 at $n = 100$ to 3.9 at $n = 300$.

The operation count cannot see any of it. Run against both widths it returns 716 steps either way, a ratio of exactly 1.00, because the number of shifts, additions and masks does not depend on how wide a field is. The saving is entirely in the size of the operands: a 217-bit field makes the row $8{,}029$ bits where a 73-bit field makes it $2{,}701$, and every addition and shift in equation (8) then moves three times as many machine words. The kernel's `Nat` operations are constant-time in the step model and linear or worse in the operand length in fact [Granlund and the GMP team, 2020].

This is worth stating as a result rather than burying as a caveat. An operation count is the natural way to reason about kernel reduction, and for this family of designs it correctly ranks list against packed and correctly predicts the order of magnitude. It is exactly wrong about the one parameter a packed representation has, so the honest procedure is the one followed here: use the count to choose between designs, and wall time to choose the width.

6.5 What the competition's own metric says

Both metrics above are substitutes, adopted because the judge's instruction counter is not reachable from outside its own harness. The judge has since scored two designs from this paper's family, which turns the substitutes into predictions that can be checked.

The entry built here, the packed row at $w = 2n + 1$, was measured at $131{,}460{,}833$ units of computation. A deliberate alternative was then built to test the opposite representational choice: the two-term recurrence of Lemma 1 computed on rows of *small* numbers, block by block so that no position is ever indexed, which performs $2{,}074$ additions across the six judged sizes against the packed design's $2{,}801$ operations while holding every value in one machine word rather than 43. It is proved for every n by the same argument, rests on the same axioms, and reduces every judged size. It was measured at $3{,}396{,}012{,}941$ units: worse by a factor of 25.8.

The ordering both substitutes predict is therefore correct, and the reason is not the one a reader might expect. Per operation the judged figures are about $180{,}000$ units for the narrow design and about $47{,}000$ for the wide one, so a one-word operation on a list costs *more* than a forty-three-word arithmetic operation, and operand width is not what the metric is charging for. What separates the designs is the kind of operation: traversing and reconstructing list cells is

	n			
Field width	100	150	200	300
$n\lceil\log_2(n{+}1)\rceil + 1$	0.08	0.16	0.32	1.06
$2n + 1$ from Corollary 3	0.03	0.06	0.11	**0.27**

Table 5: Wall time in seconds for the packed design at two field widths, measured beyond the judged range because the judged sizes are too fast to separate. The tighter width follows from Proposition 2 and is worth about a factor of 3.4. The operation count of Table 3 is identical for both rows, which is the disagreement Section 6.4 is about. Unlike Table 4 these two rows were timed through the elaborator's definitional equality checker rather than the kernel. The two routes agree on this design – the packed row totals 0.07 seconds either way across the six judged sizes – and they diverge only on the list design, so the comparison between two packed widths is unaffected; it is recorded here because a measurement should say which instrument produced it.

expensive, and arbitrary-precision arithmetic is cheap even when the operands are wide. This is the same thing Table 4 reports and the step model of Table 3 half reports, since the step model charges a list match and a `Nat` primitive alike.

Neither substitute predicts the constant. Wall time puts the ratio at 16.3 against the judged 25.8, low by a factor of 1.6; the step count puts it at 6.7, low by a factor of 3.9. Wall time is the better predictor of the two, which is the reverse of what the width study above would suggest, and the honest summary is that both substitutes rank designs reliably and neither sizes the gap. One consequence bounds Section 6.4 in turn: because the judge charges so little for width, whether the factor of 3.4 in Table 5 is worth anything on the judged metric is *not* established by anything measured here, and no judged measurement of the two widths exists.

7 The same question asked of all eight problems

The design above was developed against one problem. Asking the same question of the other seven turned out to be worth more than the design, because it found something the packing does not address and nothing in the shipped material reveals.

7.1 Two of the eight reference starters cannot be reduced at all

The challenge ships a reference starter for every problem, each of which already discharges the required correctness theorem. Timing them is therefore the obvious first step, and for six of the eight it gives what one expects: a number, sometimes comfortable and sometimes tight.

For two it gives no number at any judged size. The `primecount` starter reduces $\mathrm{impl}\, n$ for $n = 0$, 1 and 2, and every larger n fails with

```
Submission.impl 3 is not definitionally equal to 2
```

The `mertens` starter reduces only $n = 0$. Both problems judge at sizes far above those cut-offs, so both starters score nothing, and not because they are slow.

The cause is the same in both, one layer apart. `Nat.minFac` dispatches even numbers by a special case and everything else to `minFacAux`, which is defined by well-founded recursion. The kernel does not unfold well-founded recursion. `primecount` meets it directly, in a primality test; `mertens` meets it through `ArithmeticFunction.moebius`, which reaches `Nat.minFac` by way of `Squarefree` and the prime factorisation. The cut-off at $n = 3$ is exactly where `minFac` stops being able to answer from its even-number case.

This is a sharper statement of Section 1's claim than the partition measurement supports on its own. There the choice of data structure changed the cost of being checked by a factor. Here the choice of *recursion scheme* changes it from a cost into an impossibility, and it does so invisibly: both starters compile, both carry correct proofs, and neither is distinguishable from the working six without running the kernel against them.

7.2 What the repairs were, and what they measured

Table 6 gives every problem. Two entries use the packed row of Section 3.3, which discharges the concern this paper previously recorded as an open question: that the representation had been demonstrated on one recurrence only. The

Problem	What was done	Starter	Entry
`partition`	packed row (Section 3.3)	57.64 s	**0.07 s**
`ca-rule110`	packed row, cyclic mask	786,432 ops → 64	
`primecount`	structural trial division	**does not reduce**	3.79 s
`mertens`	structural factorisation	**does not reduce**	4.22 s
`sha256`	schedule fused into rounds	59.60 s	53.34 s
`permanent`	sparse rows	27.12 s	**8.79 s**
`fib`	nothing: measured competitive	2.14 s	*unchanged*
`polydisc`	nothing: measured competitive	1.51 s	*unchanged*

Table 6: Every problem in the challenge, measured the same way. Times are the worst judged case, with starter and entry timed in the same continuous integration job on the same runner, because wall time on a shared runner varies by about twenty per cent between runs: the same `sha256` starter measured 47.89 and 59.60 seconds on two runs of identical code. Comparisons across rows are therefore not meaningful, and none of these is the competition's own metric, which is a hardware instruction count. The `partition` figure is a total over six sizes rather than a single case, and `ca-rule110` is quoted in operations because its judged sizes finish inside the timer's resolution.

`ca-rule110` automaton is not a dynamic program, and its row is a cyclic neighbourhood rather than a strided sum, yet the same move applies, with the mask playing the part the width bound plays here.

Two problems were measured and deliberately left alone. Mathlib's `Nat.fastFib` reduces every judged case with about 0.36 seconds of kernel work at $n = 150{,}000$, a 31,348-digit answer; `polydisc` reduces in under two seconds. Reporting that is part of the result: a method paper that rewrites everything it touches cannot distinguish the cases where its method was needed from the cases where it was not.

7.3 An expectation that the measurement contradicted

The `sha256` entry is recorded here because it went the other way. Its starter builds a 64-word message schedule and zips it into 64 pairs for every block, roughly $65{,}000$ list cells across the judged chain, purely to drive a loop whose trip count is fixed. Removing both lists, by walking the schedule as the rounds consume it, was expected to be worth a large factor. It is worth about eleven per cent.

The measurement is the correction. This problem's cost is the round function itself, some half a million masked 32-bit operations, and not the allocation around it. The general lesson is the one Section 6.4 makes about field width from the other direction: an argument from counting allocations is no better than an argument from counting operations unless the thing counted is where the time goes.

8 Related work

The kernel, and why its cost is the cost. Lean 4's design and its kernel are described by de Moura and Ullrich [2021]; the type theory the kernel implements, and an independent implementation of it in Lean itself, are the subject of Carneiro [2024]. The reason a small kernel is worth paying for is the trusted computing base argument [Thompson, 1984]: a proof is only as trustworthy as the program that checked it, so that program should be small enough to audit. The Lean Kernel Arena [Lean FRO, 2026] makes the consequence concrete by running a dozen independent checkers against a shared suite, and reading what each declines to implement is a fast way to learn where the real difficulty sits. This paper is on the other side of the same boundary: rather than making the checker faster, it makes the term cheaper to check.

Computation inside a proof. Discharging a goal by computing is an old technique. Boutin [1997] named reflection as a way to build certified decision procedures, and Barendregt and Barendsen [2002] set out the trade in terms of autarkic against sceptical computation: either the checker does the work and you trust only the checker, or an external tool does the work and you trust its output too. Lean's `native_decide` is the sceptical option and is what this task forbids [Carneiro, 2024], which is what makes the cost of the autarkic option worth measuring. Gonthier [2008] is the largest example of the autarkic route, and the place where the cost of kernel reduction became a design constraint rather than an afterthought.

Choosing a representation for computation in a proof assistant. The idea that a verified program should be refined onto an efficient representation, with the proof carried across the refinement, is worked out for Coq by Dénès et al.

[2012]; Magaud and Bertot [2002] is the same move in miniature and the closest antecedent for this paper, since replacing a unary numeral with a binary one is exactly replacing a structure whose access cost is a distance with one whose access cost is an exponent. The alternative discipline is to verify inside the assistant and then extract to a compiled language [Haftmann and Nipkow, 2010], which trades the kernel's guarantee for the extractor's. Neither line packs a whole table row into one integer, and the reason appears to be that outside a kernel-reduction setting there is little to gain from it.

Bit-parallel dynamic programming. Carrying several table entries in the fields of one machine word, so that a strided sum becomes a shift and an add, is a known idiom and is not claimed here as new. Baeza-Yates and Gonnet [1992] introduced it for text search, and Myers [1999] gave the form closest to this one, a dynamic programming row held in one bit vector with the row step written as word arithmetic. Hyyrö [2003] tightened the analysis. What is specific to this paper is the setting rather than the trick: those algorithms pack into a fixed-width machine word and are bounded by it, whereas `Nat` is arbitrary-precision, so the row can be as long as the problem needs and the binding constraint moves from the word size to a provable bound on the entries. That is why Proposition 2 has no counterpart in the bit-parallel literature, and why it is the substance of the contribution rather than an appendix to it.

Partition counting. The partition function has far better algorithms than the specification used here, from Euler's pentagonal number recurrence to the Hardy–Ramanujan–Rademacher series [Knuth, 2011]. None of them is available: the competition locks the recurrence and asks for a proof against it, so the task is to evaluate a particular specification cheaply, not to count partitions cheaply. This is the usual situation in verified computation, where the specification is given and the freedom is in the implementation.

The author's related work. The representational question in this paper has a direct counterpart in Thakur [2026a]: there a digit is described either by its index in a sequence, which moves when the string is padded, or by its place value, which does not, and the second description is what survives a change of length. A packed row is the same construction with the roles reversed, since the field index in equation (7) *is* a place value, and it is what makes the access cost independent of the row's length. Thakur et al. [2022] applies the same idea to space, addressing a point by recursive subdivision so that each digit of the address names a scale rather than an offset. The modulus in equation (8) is wrap-around arithmetic, the setting of Thakur and Satish [2022], used here to discard the fields the shifts have overrun. Thakur [2026b] is the closest methodological relative: it bounds what a classifier can separate before any data is collected, where this paper bounds what a field can hold before any row is computed, and in both the bound is the result rather than a precaution. Thakur [2026d] is the performance-engineering counterpart, deriving its constants from a measured pipeline rather than tuning them, and identifying a class from a computed structural signature rather than a learned one, which is what the peeling identity does for the bound here. Thakur [2026c] is the same competition family and the same obligation, a verdict that only counts once the kernel has accepted a certificate for it. Three earlier papers stand in deliberate contrast, since each rests on approximation where this rests on reduction: Thakur and Satish [2021a] and Thakur et al. [2021a] train objectives whose outputs are judged by appearance, Thakur et al. [2021b] reasons under graded confidence, and Thakur and Konde [2021] supplies the background those assume. A packed field admits no graded answer at all: one bit of carry is a wrong count. Thakur and Satish [2021b] and Thakur and Talele [2026] are the closest in one respect that matters here, since both are judged outside the domain they were fitted on, and equation (5) is the proved version of that ambition: correctness at every n, not at the sizes that were tried.

9 Limitations

Five things this paper does not establish. Section 10 says which of them are close to being fixed and which is not.

The ranking metric was never measured. Every number here is a step count or a wall time. The competition ranks on hardware instruction counts, and no result in this paper establishes what the packed design scores on that metric. The step model is uncalibrated and the wall times come from a virtualised runner. What is supported is the ordering of the three designs and the order of magnitude between them; what is not supported is any specific instruction count.

Two problems, not one, and still not a general claim. An earlier version of this paper recorded the single-recurrence scope as its first limitation, since the representation had been demonstrated on one specification only. Section 7 discharges part of that: the same packing carries to `ca-rule110`, whose row is a cyclic neighbourhood rather than a strided sum and whose problem is not a dynamic program at all. Two instances are not a general claim, and the remaining six problems in the challenge were repaired by other means or left alone, so what is established is that the move is not specific to partitions, not that it applies wherever a table does.

The bound is loose and the width is not optimal. Section 4.3 measures the looseness: the proved width reserves between 3.6 and 4.9 times the bits any cell needs, and Proposition 4 would recover a factor of 1.7 of that but is not proved in the submitted file. Even it is 2.8 times wider than necessary. The tightest possible width would need the asymptotics of the partition function, a considerably larger formal development, and Section 6.4 is the reason none of this is urgent: width is invisible to the operation count and worth a constant factor in wall time.

The sum is taken one term at a time. Equation (8) accumulates $\lfloor n/d \rfloor$ shifted copies in sequence. Doubling the stride at each step would reach the same total in $\log \lfloor n/d \rfloor$ operations rather than $\lfloor n/d \rfloor$, which is worth about a factor of 2.6 on the operation count at $n = 36$. The proof obligation changes shape, since the invariant is then about a partial sum over a range of multiplicities rather than about one row, and that is why it is not in the submitted entry.

The unreducible starters are a finding about this challenge, not about Lean. Section 7.1 reports that two of the eight shipped reference starters cannot be reduced by the kernel at any judged size. That is a fact about those two definitions reaching well-founded recursion, established by running them. It is not a claim that well-founded recursion is always fatal, nor a measurement of how common the situation is beyond these eight.

One prediction in this paper was wrong by a factor. Section 7.3 records an expected large gain from removing about 65,000 list allocations that measured at eleven per cent. It is left in because the correction is the useful part, and because a paper that reports only its successful predictions gives no way to calibrate the unsuccessful ones.

The step model is the paper's own instrument. The operation counts come from a transcription of the Lean definitions into Python, written by the same author as the definitions. A transcription error would move the numbers without being caught by the correctness check, which only verifies the answer. The mitigation is that the transcription is short, public and re-run in continuous integration, not that it has been independently validated.

10 What remains

The three open items are stated in Section 9 rather than promised here, but two of them are close. The logarithmic-stride sum is a mechanical change to sweep with a harder invariant, and the tighter bound is the same induction as Proposition 2 with a different arithmetic step. The third, applying the representation to the other seven problems, is the one that would turn a single measurement into evidence about a method, and it is the only one that would change what this paper is entitled to claim.

11 Conclusion

The kernel charges for what it has to reduce, and a table row held as a list makes it reduce a distance. Holding the row as the digits of one natural number removes the distance: the entry for m is found by a shift and a mask, and the recurrence's sum over multiplicities becomes a handful of shifts and additions performed on the entire row at once. On the six judged sizes that is 2,801 primitive steps against 182,642 for the lists and 1,112,856 for the specification as written, and 0.07 seconds of reduction wall time against 57.64. The competition's own metric, which was unavailable when the designs were chosen, has since scored this entry at 131,460,833 units against 3,396,012,941 for a deliberately narrow-operand alternative built to test the representational choice, confirming the ordering by a factor of 25.8.

What makes it a result rather than a trick is that the packing has a precondition and the precondition is proved. Fields corrupt each other silently if an entry overflows, and no amount of testing at the judged sizes would establish that none does, because the statement to be proved is universal. Peeling one term off the specification's own sum gives a two-term identity, the identity gives $\mathrm{partAux}\, k\, m \le 2^{m+k}$ by a nested induction, and that bound fixes the field width at $2n + 1$ bits. The whole development closes in core Lean on `propext` and `Quot.sound`.

The measurement also marks its own instrument. An operation count ranks the three designs correctly and is blind to field width, which is worth a further factor of 3.4 that only wall time sees. Counting operations is the right way to choose a representation for kernel reduction and the wrong way to tune one, and a paper that reported only the metric flattering to its method would have missed the second half of that sentence.

Artefacts

Everything in this paper is reproducible from https://github.com/Amey-Thakur/SAIR-LEAN-KERNEL-CHALLENGE: the submitted Lean file, the earlier list-based entry kept for comparison,

the narrow-operand alternative of Section 6.5, the benchmark harness and the continuous-integration workflow that produced Table 4. The measurements behind every number quoted above are in `paper-data/` of the same repository: the step model of Table 3, the timings of Tables 4 and 5, the bound slack of Section 4.3, the eight-problem summary, and the judge's own figures of Section 6.5. The same directory carries the guard that reads this manuscript and fails if any quoted figure disagrees with those files, which is how the two are kept from drifting apart.

Acknowledgements

The specification, the axiom rules and the test structure are the competition's own [Breitner et al., 2026]. No item from the Contributor Network and no external source was used in constructing the entry.

References

J. Breitner, L. de Moura, K. Morrison, and T. Tao. SAIR Lean Kernel Challenge, Stage 1. The Lean FRO and the SAIR Foundation, deadline 20 November 2026. https://competition.sair.foundation/competitions/lean-kernel-challenge. Fixes the specification of equations (1) to (4), the permitted axioms, the prohibition on `native_decide`, and hardware instruction count as the rank. Every constraint this paper works under is set there.

L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction, CADE 28*, pages 625–635, 2021. 10.1007/978-3-030-79876-5_37. Describes the system and the kernel whose reduction cost this paper measures, including the decision to implement `Nat` arithmetic as a primitive on arbitrary-precision integers, which is what makes a packed row one operation rather than many.

M. Carneiro. Lean4Lean: verifying a typechecker for Lean, in Lean. arXiv:2403.14064, 2024. The precise account of what Lean's kernel accepts and of what an added axiom costs, which is the standard against which `#print axioms` in Section 5 is read, and the reference for why `native_decide` is a different kind of step from a reduction.

The Lean FRO. The Lean kernel arena. 2026. https://arena.lean-lang.org/. https://github.com/leanprover/lean-kernel-arena. Runs a dozen independent kernel implementations against a shared suite. Read here for what each declines to implement, which locates the expensive parts of checking and so the parts a submission should avoid generating.

K. Thompson. Reflections on trusting trust. *Communications of the ACM*, 27(8):761–763, 1984. 10.1145/358198.358210. The argument for keeping the checked-by program small, which is why the kernel does not memoise and therefore why the cost this paper reduces exists at all.

S. Boutin. Using reflection to build efficient and certified decision procedures. In *Theoretical Aspects of Computer Software*, pages 515–529, 1997. 10.1007/BFb0014565. Establishes computing inside the assistant as a proof technique. The present entry is an instance: the proof of equation (5) is small and the reduction is where the work goes.

H. Barendregt and E. Barendsen. Autarkic computations in formal proofs. *Journal of Automated Reasoning*, 28(3):321–336, 2002. 10.1023/A:1015761529444. Names the choice between having the checker do the work and trusting an external tool that did it. The competition takes the first option by rule, which is what makes the cost of reduction the quantity worth optimising.

G. Gonthier. Formal proof: the four-colour theorem. *Notices of the American Mathematical Society*, 55(11):1382–1393, 2008. The largest computation carried out inside a kernel, and the work that made reduction cost a design constraint rather than an implementation detail.

M. Dénès, A. Mörtberg, and V. Siles. A refinement-based approach to computational algebra in Coq. In *Interactive Theorem Proving*, pages 83–98, 2012. 10.1007/978-3-642-32347-8_7. Carries a proof across a change of data representation, which is the general form of what Section 3.3 does for one row of one table.

N. Magaud and Y. Bertot. Changing data structures in type theory: a study of natural numbers. In *Types for Proofs and Programs*, pages 181–196, 2002. 10.1007/3-540-45842-5_12. Replaces a unary numeral with a binary one and moves the proofs across. The closest antecedent here, because it is the same exchange this paper makes at the level of a table: an access cost that is a distance for one that is an exponent.

F. Haftmann and T. Nipkow. Code generation via higher-order rewrite systems. In *Functional and Logic Programming*, pages 103–117, 2010. 10.1007/978-3-642-12251-4_9. The alternative discipline: verify inside the assistant, then extract and run outside it. Cited as the option this task forbids, and as the reason the forbidding is substantive.

R. Baeza-Yates and G. H. Gonnet. A new approach to text searching. *Communications of the ACM*, 35(10):74–82, 1992. 10.1145/135239.135243. Introduces carrying many states in the bits of one word so that a transition becomes a shift and a mask. The idiom equation (8) uses, credited rather than claimed.

G. Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*, 46(3):395–415, 1999. 10.1145/316542.316550. The closest algorithmic relative: a dynamic programming row held in one bit vector, with the row step written as word arithmetic. The difference is the bound, since a machine word fixes the field width from outside while `Nat` requires Proposition 2 to fix it from inside.

H. Hyyrö. A bit-vector algorithm for computing Levenshtein and Damerau edit distances. *Nordic Journal of Computing*, 10(1):29–39, 2003. Tightens the analysis of the bit-vector row step and is the reference for how the packing behaves once the row exceeds one word, which is the regime every row in this paper is in.

T. Granlund and the GMP development team. GNU multiple precision arithmetic library, version 6.2. 2020. https://gmplib.org/. The library behind Lean's `Nat` primitives. Its cost model, linear or worse in operand length, is why the field-width study of Table 5 finds a factor the step count cannot see.

D. E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. Addison-Wesley, 2011. The reference for partition counting, and for the better recurrences this paper is not permitted to use. Cited to make the constraint explicit: the task is to evaluate one fixed specification cheaply, not to count partitions cheaply.

A. Thakur. Place value, not position: length generalisation limits in neural modular arithmetic. 2026. https://github.com/Amey-Thakur/SAIR-MODULAR-ARITHMETIC-CHALLENGE. Finds that describing a digit by its place value rather than its index is what survives a change of length. A packed row is that same description used for a table: the field index in equation (7) is a place value, which is why the access cost does not grow with the row.

A. Thakur. Chebotarev fingerprints: identifying degree-24 Galois groups without computer algebra. 2026. https://github.com/Amey-Thakur/SAIR-INVERSE-GALOIS-PROBLEM-IGP24. Bounds what a classifier can separate before any data is collected. The methodological relative of Proposition 2, which bounds what a field can hold before any row is computed; in both the bound is the result rather than a safeguard around it.

A. Thakur. The verdict is free: a saturated benchmark for equational implication, and what it leaves to prove. 2026. https://github.com/Amey-Thakur/SAIR-MATHEMATICS-DISTILLATION-CHALLENGE. The complementary half of the same question, in the same competition family and under the same judge. There a lookup against a public lattice settles every verdict, so the difficulty is entirely in producing a certificate; here the certificate is easy to state and the difficulty is entirely in making the kernel able to check it. Both papers end at the same place from opposite directions: an answer counts only once a kernel has accepted an artefact for it, and which half is hard is a property of the problem rather than of the solver.

A. Thakur. Frame-synchronous hand gesture detection by projected winding order. arXiv:2609.13269, 2026. 10.48550/arXiv.2609.13269. Identifies a class from a computed structural signature rather than a learned representation, and derives its pipeline constants from measurement rather than tuning them. Both moves recur here, in the peeling identity and in the field width.

A. Thakur, M. Satish, R. K. Kahlon, H. Rizvi, and A. Davare. QuadTree visualizer. *International Journal of Engineering Research and Technology*, 11(4), 2022. 10.5281/zenodo.18447415. Addresses a point by recursive subdivision, so each digit of the address names a scale rather than an offset. The spatial form of the packing in equation (7).

A. Thakur and M. Satish. Clock synchronization in distributed systems. *International Research Journal of Engineering and Technology*, 9(3), 2022. https://www.irjet.net/archives/V9/i3/IRJET-V9I3350.pdf. Wrap-around arithmetic handled explicitly, which is what the modulus in equation (8) does to discard the fields the shifts overran.

A. Thakur and M. Satish. Generative adversarial networks. *International Journal for Research in Applied Science and Engineering Technology*, 9(8):2307–2325, 2021. 10.22214/ijraset.2021.37723. Cited for contrast. An adversarial objective is judged by whether its output looks right; a packed field is judged by whether one bit carried, and admits no approximate answer.

A. Thakur, H. Rizvi, and M. Satish. White-box cartoonization using an extended GAN framework. *International Journal of Engineering Applied Sciences and Technology*, 5(12), 2021. 10.33564/IJEAST.2021.v05i12.049. Decomposes a problem into representations before operating on it, and finds that what transfers is decided by the decomposition rather than by capacity. The same claim this paper makes about where reduction cost comes from.

A. Thakur, K. Dhiman, and M. Phansikar. Neuro-fuzzy: artificial neural networks and fuzzy logic. *International Journal for Research in Applied Science and Engineering Technology*, 9(9):128–135, 2021. 10.22214/ijraset.2021.37930. Reasoning under graded confidence, included as the opposite regime: equation (5) is an equality proved for every n, with no degree of membership available.

A. Thakur and A. Konde. Fundamentals of neural networks. *International Journal for Research in Applied Science and Engineering Technology*, 9(VIII):407–426, 2021. 10.22214/ijraset.2021.37362. The background the author's learning-based papers assume, and the contrast that locates this one: there are no fitted parameters anywhere in the present entry, so every quantity in it is either defined or proved.

A. Thakur and M. Satish. Adversarial open domain adaption framework (AODA): sketch-to-photo synthesis. *International Journal of Engineering Applied Sciences and Technology*, 6(2), 2021. 10.33564/IJEAST.2021.v06i02.037. Performance outside the domain the model was fitted on, measured. Equation (5) is the proved version of the same ambition: correctness at every n, not at the sizes that were tried.

A. Thakur and S. Talele. A modular zero-shot pipeline for accident detection, localization, and classification in traffic surveillance video. arXiv:2604.09685, 2026. 10.48550/arXiv.2604.09685. Composes stages with no task-specific training, so the guarantee comes from what each stage already establishes. The same compositional reading applies here, with the stages being lemmas and the guarantee being the kernel's.